

Concepto 2

Spring Security.

Importando de los archivos que vimos esta semana 3, explicare sobre lo que es el tema de spring security. Partiendo de la idea que como demuestras a un servidor que realmente eres tu el que esta intentando leer, modificar, crear o borrar data, llegamos a lo que es la autenticación y autorización de usuarios, es decir, quien eres y que puedes hacer. Estos son 3 pasos, el envío de credenciales, el control de autenticación y el control de autorización.

Iniciamos con el protocolo mas sencillo, que es el de HTTP basic, donde el usuario tiene que mandar la contraseña que vez que quiera acceder al servidor. Asumiendo un servidor con base de datos mysql con tablas de miembros y de roles y contraseñas en algoritmo Bcrypt (el propio folder incluye los sql scripts para crear y poblarlo), veremos los proyectos cada uno con diferentes implementaciones de seguridad. Con el proyecto `..\02 Spring Security\17-seguridad-autenticacion\01-security-basic` implementando este protocolo, que es una implementación con seguridad de nuestro pasado proyecto 16, las piezas importantes modificados son la adición de spring security en el pom.xml, un archivo de java adicional que maneja la seguridad en Securityconfig.java, y ajustes en el application.properties.

Con la dependencia de spring boot starter security, nos deja que spring intercept peticiones de http y aplicar seguridad antes de que llegue al REST controller del proyecto, siendo guiado por el securityconfig.java.

En este archivo de código java, se definen dos metodos @Bean: userDetailsService y filterChain. Como los comentarios adentro del código dice, el primero es para saber quien es el usuario, y el segundo que pueden hacer. Al ser beans, spring los crea y maneja como parte de la aplicación.

UserDetailsService pregunta la información del usuario, basándose en el JDBCUserdetailsmanager, definiendo que la información de autenticación lo pasa la base de datos. Luego, pasamos la datasource que es la conexión a base de datos inyectada por spring. Con todo esto, al intentar autenticar un usuario buscamos que se encuentre en la tabla de miembros, el ? siendo sustituido por el parámetro mandado en la petición. Luego, preguntamos que roles tiene nuestro usuario, siendo regresado que roles específicos este usuario, si es válido, tiene. En nuestra base de datos mysql, si lo observas a través de cualquier viewer apropiado, en mi caso Dbeaver, vemos que los passwords de los nuestros miembros están encriptados con Bcrypt, siendo soportados con el spring security. Spring security no desencripta el password, solo hace match con el hash guardado y si pasa, lo autentica, sino, regresa 401.

Pasando a la parte que maneja la API de nuestro proyecto, el segundo bean filterchain crea esta cadena de seguridad, creando revisiones por requests.

¿Quieres un get individual o de todos? Necesitas el rol de employee, guardado en la base de datos como ROLE_EMPLOYEE. Para POST, PUT y PATCH, necesitas tener el rol de manager, y DELETE necesitas el de admin. Al último, si por alguna razón nuestro request no es ninguno de los que definimos, comoquiera requerimos que tenga autenticación.

La línea de comando que define que usemos HTTP es el `http.httpBasic(Customizer.withDefaults())`, diciéndole a spring que use este protocolo, y hacemos ajustes como dicen los comentarios en código para desactivar csrf y de tener una sesión stateless, forzando que cada petición se autentique.

Para correr el proyecto, solo hay que correr en Windows powershell:

```
cd "C:\...\17-seguridad-autenticacion\01-security-basic"
```

```
.\mvnw.cmd spring-boot:run
```

Para levantarlo en una terminal, y luego en otra hacer los requests apropiados.

Hay que recordar que la debilidad en este proyecto es el hecho de que, en cada petición, mandamos la contraseña sin seguridad, solo empaquetado en base64. Aunque la API esta asegurada teóricamente, si alguien observa lo que mandamos la seguridad se rompe. Si quisiéramos arreglar esto, deberíamos usar https.

En el siguiente proyecto que es \02-security-jwt, inicializando de la misma manera solo que cambiando el folder a la mencionada aquí y la aplicación siendo levantada en puerto 8072, cambiamos el hecho que cada petición incluía las propias credenciales, mientras que en JWT solo se usa en la operación de login. Se agrega en el pom.xml el spring starter security oauth2 resource server, dando a spring security las herramientas de JWT y de Oauth2. Por el momento nos enfocamos en JWT, que es un formato de tokens.

Vemos que nuestro SecurityConfig.java cambio, con una gran adición de importaciones de paquetes, y el manejo de las llaves RSA privadas y públicas, donde la llave privada es para el log in y la llave publica para la verificación. Siguiendo la misma base de datos y casi ningún cambio en userdetailsservice, vemos la creación de dos cadenas de filtro, la de log in inicial y la del token JWT de verificación. Usando como base el protocolo http básico para autenticar este primer paso, se sigue la lógica de usuario y contraseña para obtener un JWT. Esto es en la cadena de filtro 1, sin sesión de log in tradicional.

Para el resto de la API, como el filtro 1 solo atrapa autenticación, filtro 2 maneja todo lo demás, y solamente se dejan pasar las peticiones REST si tienen tokens JWT. Para conseguir esos tokens, se tiene la sección de JwtEncoder, que firma los tokens con las llaves privadas RSA. Su contraparte es JwtDecoder, que verifica los tokens con las llaves públicas. La parte del código de JwtAuthenticationConverter es solo para pasar del JWT al formato de spring.

De aquí, pasamos a AuthController.java, que toma un usuario autenticado y lo convierte a JWT, donde se inyecta el JwtEncoder. De ahí pasamos al método login que ya no pide autenticación, manejado ya previamente. De aquí, se crea un objeto para autenticar incluyendo el nombre de usuario y sus roles por parte de spring, la creación del token tomando el tiempo a ese instante y poniendo el limite de tiempo. El token al final va firmado, no cifrado, no guarda datos secretos. Esto, junto con el limite ttl incluido en nuestro application.properties, nos da una sesión de 1 hora.

De aquí puedes tener varias pruebas, al inicializar

```
$TOKEN = (curl.exe -s -u "john:test123" -X POST  
"http://localhost:8072/api/auth/login" | ConvertFrom-Json).accessToken
```

```
curl -H "Authorization: Bearer $TOKEN" -X DELETE  
http://localhost:8072/api/employees/1
```

```
{"status":403,"error":"Forbidden"}
```

Deberia ser el resultado para una prueba rápida.

En nuestro \03-security-oauth2, se aplica la delegación, ya que se remuevan grandes partes de código de autenticación en la aplicación, y hacemos Keycloak responsable del manejo de usuarios, contraseñas y demás, nuestro api solo valida tokens y roles. Manejando que nuestro application properties si tenga nuestro jwt issuer para oauth2, en nuestro caso el localhost 8090 si lo levantaste en docker, siendo el Keycloak realm de academy. Se incluyen los scripts en el folder para keycloak, y ponemos a andar nuestro proyecto en puerto 8073.

De manera muy resumida le decimos a spring que se conecte a keycloak, con la configuración de openID, nuestro archivo de seguridad java se redujo, manteniendo la parte de autorización igual y la autenticación con oauth2, simplificando nuestra aplicación como un resource server y autenticando peticiones entrantes con JWT bearer tokens. Modificamos la respuesta de keycloak para que este de acorde al formato de spring security con el keycloakconverter() y extraerroles(). Ahora nuestros usuarios viven en keycloak, y asumimos que nuestra base de datos solo contiene los employees data que trabajamos antes, por que ya no necesitamos manejar nosotros nuestros usuarios.

También se incluye en el folder los scripts para testing y levantamiento del keycloak.